

E-Commerce Ecosystem System Design Specification

Version: 1.0

Classification: Internal Architecture Specification

Architecture Style: Modular Monolith with Bounded Context Integration

Primary Language: Java 21

Framework: Spring Boot

Persistence: MySQL

Security: JWT + Capability-Based RBAC

Documentation Standard: OpenAPI / Swagger

Integration Style: Synchronous REST Communication

1. System Overview

1.1 Purpose

The E-Commerce System is responsible for managing the complete customer purchasing journey, including:

- Product catalog management
- Customer account management
- Authentication and authorization
- Shopping cart management
- Checkout orchestration
- Order lifecycle management
- Payment orchestration
- Media management
- External inventory integration

The platform is designed as part of a larger E-Commerce Ecosystem and operates independently from the Inventory Management System (IMS).

The E-Commerce System owns customer-facing commerce workflows, while inventory ownership remains delegated to the Inventory System.

1.2 Architectural Principles

The architecture follows:

- Domain-Driven Design (DDD)
- Clean Architecture
- Modular Monolith Architecture
- Bounded Context Separation

- Capability-Based Authorization
 - Actor-Centric Identity Model
 - Anti-Corruption Layer Integration
 - API-First Design
 - OpenAPI Documentation Standardization
 - Auditability and Observability by Design
-

1.3 System Responsibilities

The E-Commerce System owns:

- Customer accounts
- Authentication
- Authorization
- Product catalog
- Product visibility
- Categories
- Media management
- Shopping carts
- Checkout
- Orders
- Order lifecycle
- Customer purchase experience
- Payment orchestration
- Order expiration

The E-Commerce System does NOT own:

- Inventory
- Stock availability
- Reservation storage
- Reservation expiration
- Stock commitment

Those responsibilities belong exclusively to the Inventory System.

2. Core Domain Concepts

2.1 Actor

Represents any business identity capable of interacting with the system.

Examples:

- Account
- Customer
- Seller

- Administrator
- Future Service Clients

Actors are represented through ActorIdentity abstractions rather than direct security principals.

2.2 Account

Represents an authenticated human identity.

Responsibilities:

- Authentication
 - Authorization
 - Profile management
 - Role assignment
 - Verification workflows
-

2.3 Capability

Represents a granular permission.

Examples:

- product_create
- product_update
- category_read
- order_cancel

Capabilities are the atomic authorization unit.

2.4 Role

Collection of capabilities.

Examples:

- Customer
- Seller
- Administrator

Roles provide RBAC-based authorization.

2.5 Category

Organizes products into business-defined catalog structures.

Categories support:

- Lifecycle management
 - Auditing
 - Soft deletion
 - Media association
-

2.6 Product

Represents a sellable catalog item.

Product lifecycle:

- Draft
- Active
- Inactive
- Deleted

Products maintain:

- Catalog metadata
- Category association
- Media association
- Inventory linkage

Inventory data is never owned by Product.

2.7 Cart

Represents customer purchase intent.

Cart characteristics:

- Temporary workspace
- Customer-owned
- Mutable
- Non-reserved

Important:

Cart does not reserve inventory.

2.8 Reservation

Represents temporary inventory allocation.

Owned by:

- Inventory System

Purpose:

- Prevent overselling
 - Support checkout workflows
-

2.9 Order

Represents a committed purchase request.

Order characteristics:

- Immutable purchase record
- Historical business artifact
- Snapshot-based design

Orders contain product snapshots rather than live product references.

2.10 Media

Centralized image management subsystem.

Supports:

- Product images
 - Category images
 - Future media types
-

3. System Components

3.1 Core Infrastructure Module

Provides platform-wide foundations.

Responsibilities:

- Exception handling
- API contracts
- Pagination
- Logging

- Auditing
 - Actor abstractions
 - Identity abstractions
-

Major Components

Exception Framework

Error categories:

- Validation
- Business
- Security
- Technical

Provides:

- Consistent error contracts
 - Centralized exception handling
-

API Infrastructure

Provides:

- Standardized responses
 - Pagination abstraction
 - Metadata handling
 - Consistent API contracts
-

Observability Infrastructure

Provides:

- Request tracing
 - MDC propagation
 - Security logging
 - Business event logging
 - Structured JSON logs
-

Auditing Infrastructure

Provides:

- Created By
- Updated By
- Created At

- Updated At

Using ActorIdentity attribution.

3.2 Identity Module

Responsible for:

- Accounts
 - Roles
 - Capabilities
 - Authentication
 - Verification
-

Subsystems

Capability Management

Provides:

- Capability registry
 - Capability publication
 - Capability discovery
-

Role Management

Provides:

- Role lifecycle
 - Capability assignment
 - Default roles
-

Account Management

Provides:

- Registration
 - Profile management
 - Role assignment
 - Account lifecycle
-

Authentication Module

Provides:

- Login
 - JWT generation
 - JWT validation
 - Password reset
 - Account activation
-

Verification Module

Provides:

- OTP verification
 - Password recovery
 - Activation workflows
-

Email Module

Provides:

- Template rendering
 - Email delivery
 - Retry processing
 - Email auditing
-

3.3 Catalog Module

Responsible for:

- Categories
 - Products
-

Category Submodule

Provides:

- Category CRUD
 - Lifecycle management
 - Auditing
-

Product Submodule

Provides:

- Product lifecycle
 - Product publication
 - Product inventory linkage
 - Catalog visibility
-

3.4 Media Module

Responsible for image lifecycle management.

Provides:

- Image storage
- Variant generation
- Public URLs
- Resolution management

Supported consumers:

- Products
 - Categories
 - Future domains
-

3.5 Cart Module

Responsible for customer shopping sessions.

Capabilities:

- Add items
- Remove items
- Update quantities
- Retrieve cart

Cart remains inventory-independent.

3.6 Order Module

Responsible for:

- Checkout

- Order placement
 - Order cancellation
 - Order expiration
 - Reservation coordination
-

Order Aggregate

Owns:

- Order state transitions
 - Business invariants
 - Payment coordination
-

Supported States

- WAITING
- CONFIRMED
- CANCELLED
- EXPIRED

Future:

- PROCESSING
 - SHIPPED
 - DELIVERED
 - REFUNDED
-

3.7 Inventory Integration Module

Anti-Corruption Layer between E-Commerce and Inventory.

Responsibilities:

- Authentication
 - Token management
 - API communication
 - Contract translation
 - Exception translation
-

Inventory Gateway

Provides:

- stockExists()

- reserveStock()
 - confirmReservation()
 - releaseReservation()
-

4. External System Integrations

4.1 Inventory Management System

Integration Type:

Synchronous REST

Responsibilities:

- Stock ownership
 - Reservation ownership
 - Reservation expiration
 - Stock commitment
-

Integration APIs

Reserve Stock

Creates temporary reservation.

Confirm Reservation

Consumes reserved inventory.

Release Reservation

Returns inventory to available stock.

Stock Validation

Verifies stock existence.

5. High-Level Data Models

Account

Fields:

- AccountCode
 - Username
 - Email
 - Status
 - Roles
-

Role

Fields:

- RoleCode
 - Name
 - Type
 - Capabilities
-

Category

Fields:

- CategoryCode
 - Name
 - Description
 - Status
-

Product

Fields:

- ProductCode
 - Name
 - Description
 - Price
 - Category
 - Status
 - StockReference
-

Cart

Fields:

- CartCode
 - Customer
 - Items
-

Cart Item

Fields:

- ProductCode
 - Quantity
-

Order

Fields:

- OrderNumber
 - CustomerIdentity
 - Status
 - Subtotal
 - ShippingCost
 - TaxAmount
 - DiscountAmount
 - TotalAmount
 - ReservationId
 - ExpiresAt
 - PlacedAt
-

Order Item

Fields:

- ProductCode
 - ProductName
 - UnitPrice
 - Quantity
 - Subtotal
-

Image

Fields:

- ImageCode
 - StorageKey
 - Variants
 - OwnerReference
-

6. End-to-End Workflows

6.1 Customer Registration Flow

1. Customer submits registration.
 2. Account created.
 3. Verification token generated.
 4. Verification email sent.
 5. Customer activates account.
 6. Account becomes active.
-

6.2 Product Publication Flow

1. Seller creates product.
 2. Product remains Draft.
 3. Product linked to inventory stock.
 4. Stock existence validated.
 5. Product published.
 6. Product becomes visible.
-

6.3 Checkout Workflow

Step 1

Customer reviews cart.

No inventory reservation exists.

Step 2

Customer enters checkout information.

No inventory reservation exists.

Step 3

Customer confirms checkout.

System validates:

- Cart
 - Quantities
 - Product availability
-

Step 4

Inventory reservation requested.

Inventory validates stock.

Failure

Reservation rejected.

Result:

- No order created
-

Success

Reservation created.

Inventory returns:

- ReservationId
 - ExpirationTime
-

Step 5

Order created.

Order contains:

- ReservationId
- ExpirationTime

Status:

WAITING

Step 6

Customer redirected to payment.

6.4 Payment Success Flow

1. Payment succeeds.
 2. Reservation confirmation requested.
 3. Inventory commits stock.
 4. Order becomes CONFIRMED.
-

6.5 Payment Failure Flow

1. Payment fails.
 2. Reservation released.
 3. Order becomes CANCELLED.
-

6.6 Abandonment Flow

1. Reservation created.
 2. Customer leaves.
 3. Reservation expires in Inventory.
 4. Inventory releases stock.
 5. Order expiration job executes.
 6. Order becomes EXPIRED.
-

7. State Machines

Product Lifecycle

Draft ↓ Active ↓ Inactive ↓ Deleted

Verification Token

Generated ↓ Active ↓ Used

or

Generated ↓ Expired

Email Lifecycle

Pending ↓ Sending ↓ Sent

or

Pending ↓ Failed ↓ Retry ↓ Sent

Order Lifecycle

WAITING ↓ CONFIRMED

or

WAITING ↓ CANCELLED

or

WAITING ↓ EXPIRED

Reservation Lifecycle (Inventory Owned)

ACTIVE ↓ CONFIRMED

or

ACTIVE ↓ RELEASED

or

ACTIVE ↓ EXPIRED

8. Consistency & Transaction Strategy

Local Transactions

Used within individual modules.

Examples:

- Account creation
 - Product creation
 - Order persistence
-

Cross-System Consistency

Uses orchestrated workflow.

Sequence:

Reserve Inventory → Create Order → Process Payment → Confirm Reservation

No distributed transaction is used.

Consistency Model

Eventual consistency across system boundaries.

Strong consistency inside individual systems.

9. Failure Scenarios & Recovery Strategy

Reservation Failure

Cause:

Insufficient stock.

Recovery:

Checkout rejected.

No order created.

Payment Failure

Cause:

Gateway rejection.

Recovery:

Release reservation.

Cancel order.

Inventory Unavailable

Cause:

Network failure.

Recovery:

Retry policy.

Technical exception generated.

Order creation blocked.

Email Failure

Cause:

SMTP outage.

Recovery:

Retry mechanism.

Scheduled recovery jobs.

Token Expiration

Cause:

User inactivity.

Recovery:

Generate new token.

Invalidate previous token.

Authentication Failure

Cause:

Invalid credentials.

Recovery:

Reject login attempt.

Audit security event.

10. Edge Cases

Product Updated After Order Creation

Order remains unchanged.

Order snapshot preserved.

Product Deleted After Purchase

Order history remains valid.

Snapshots prevent historical corruption.

Customer Pays After Reservation Expiration

Reservation confirmation fails.

Order transitions to EXPIRED or CANCELLED.

Manual recovery workflow may be required.

Inventory Reservation Created But Order Creation Fails

Reservation must be released through compensation workflow.

Duplicate Payment Callback

Order state transition must be idempotent.

Repeated confirmation requests must not duplicate inventory commitment.

Multiple Concurrent Checkouts

Inventory reservation prevents overselling.

First successful reservation wins.

Cart Contains Inactive Product

Checkout validation fails.

No reservation created.

Customer Abandons Payment Page

Reservation expires automatically.

Order expires automatically.

Inventory becomes available again.

11. Architectural Outcomes

The E-Commerce System establishes a complete customer commerce platform built around domain-driven boundaries, capability-based security, actor-centric identity management, inventory decoupling, and resilient order orchestration.

The architecture preserves strict ownership boundaries between Commerce and Inventory domains while enabling secure, scalable, and maintainable business workflows suitable for future expansion into payments, shipping, fulfillment, marketplaces, promotions, and distributed ecosystem services.